

Lab Assignment 01

Predicting Diabetes with Multilayer Perceptrons

1. Problem Statement

Develop a binary classification system for predicting whether a patient is likely to have diabetes using clinical and demographic attributes. The project should implement and compare multiple machine learning/deep learning classification models, followed by the development of a Multilayer Perceptron (MLP) model.

Students must perform Exploratory Data Analysis (EDA), data preprocessing, feature engineering, model development, evaluation, hyperparameter tuning, and model deployment. The final model should be selected based on appropriate evaluation metrics rather than simply choosing the model with the highest accuracy.

2. Aim

To develop, evaluate, optimize, and deploy a Multilayer Perceptron-based diabetes prediction system using a suitable diabetes dataset.

3. Objectives

- [1] Understand the structure and characteristics of a diabetes dataset.
- [2] Perform systematic EDA to identify patterns, distributions, correlations, and anomalies.
- [3] Identify and handle missing, inconsistent, and potentially erroneous values.
- [4] Perform appropriate data preprocessing and feature scaling.
- [5] Develop multiple classification models for comparison.
- [6] Design and train an MLP for diabetes prediction.
- [7] Compare the performance of different models using appropriate classification metrics.
- [8] Tune the hyperparameters of the selected MLP architecture.
- [9] Analyze overfitting and underfitting using training and validation performance.
- [10] Save the trained model and develop a simple deployment interface.
- [11] Interpret the results and discuss limitations of the developed system.

4. Dataset

Students may use the Pima Indians Diabetes Dataset or another instructor-approved diabetes dataset.

Recommended Dataset

Pima Indians Diabetes Database

- Binary classification problem
- Target variable: Outcome (0 → Non-diabetic; 1 → Diabetic)
- Typical features: Pregnancies, Glucose, Blood Pressure, Skin Thickness, Insulin, BMI, Diabetes Pedigree Function, Age

Dataset source: <https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database>

Students must clearly document:

- Dataset source
- Number of observations

- Number of features
- Target variable
- Feature descriptions
- Data types
- Class distribution
- Known limitations or biases

5. Dataset Understanding

Students should perform an initial investigation of the dataset.

- ☐ Load the dataset using Pandas.
- ☐ Display the first and last few records.
- ☐ Determine the number of rows and columns.
- ☐ Identify numerical and categorical features, if present.
- ☐ Display data types.
- ☐ Generate descriptive statistics.
- ☐ Identify the target variable.
- ☐ Determine the number of samples in each target class.
- ☐ Prepare a brief description of each feature.

Expected Output

Feature	Data Type	Description	Role
Glucose	Numerical	Plasma glucose concentration	Predictor
BMI	Numerical	Body mass index	Predictor
Age	Numerical	Patient age	Predictor
Outcome	Binary	Diabetes status	Target

6. Exploratory Data Analysis (EDA)

Perform comprehensive EDA before developing the models.

6.1 Univariate Analysis

- ☐ Distribution of each numerical feature.
- ☐ Mean, median, standard deviation, minimum and maximum.
- ☐ Histograms.
- ☐ Box plots.
- ☐ Skewness and potential outliers.

6.2 Target Analysis

- ☐ Calculate the number of diabetic and non-diabetic patients.
- ☐ Calculate class percentages.
- ☐ Visualize class distribution using a bar chart or count plot.
- ☐ Discuss whether class imbalance exists.

6.3 Bivariate Analysis

Investigate relationships such as Glucose vs Outcome, BMI vs Outcome, Age vs Outcome, Blood Pressure vs Outcome, and Pregnancies vs Outcome.

- ☐ Box plots
- ☐ Violin plots
- ☐ Scatter plots

- ☐ Grouped statistics

6.4 Correlation Analysis

- ☐ Generate a correlation matrix.
- ☐ Create a heatmap.
- ☐ Identify strongly correlated features.
- ☐ Discuss whether multicollinearity may affect the models.

6.5 Outlier Analysis

- ☐ IQR method
- ☐ Box plots
- ☐ Z-score, where appropriate

Students should not automatically delete every detected outlier. They must justify whether an observation should be removed, transformed, or retained.

7. Data Quality and Preprocessing

- ☐ Check for missing values.
- ☐ Identify invalid or physiologically implausible values.
- ☐ Decide how missing/invalid values should be handled.
- ☐ Check for duplicate records.
- ☐ Handle outliers where justified.
- ☐ Encode categorical variables, if applicable.
- ☐ Separate features X and target y.

Important Requirement: Students should investigate technically valid but potentially implausible or suspicious values before modeling rather than automatically treating them as valid.

8. Feature Engineering

Students should investigate whether additional meaningful features can improve prediction.

- ☐ Age groups
- ☐ BMI categories
- ☐ Glucose-related categories
- ☐ Interaction features
- ☐ Log transformation of highly skewed variables
- ☐ Feature selection based on correlation or model importance

Students must explain the rationale behind every engineered feature.

9. Train-Test Split

Recommended approach: Training set 70–80%; Testing set 20–30%.

- ☐ Use a fixed random seed.
- ☐ Use stratification when appropriate.
- ☐ Ensure preprocessing operations such as scaling are fitted only on training data.
- ☐ Apply the fitted transformation to validation/test data.

10. Feature Scaling

Investigate and apply an appropriate feature scaling technique.

- Standardization using StandardScaler
- Min-max normalization using MinMaxScaler
- ☐ Explain why scaling is important for an MLP.
- ☐ Apply the selected scaling technique.
- ☐ Compare feature distributions before and after scaling.

11. Baseline and Multiple Model Development

Students must not directly select the MLP without comparison. Develop at least three classification models, including the MLP.

- i. Logistic Regression
- ii. K-Nearest Neighbors
- iii. Support Vector Machine
- iv. Decision Tree
- v. Random Forest
- vi. Gradient Boosting
- vii. Multilayer Perceptron

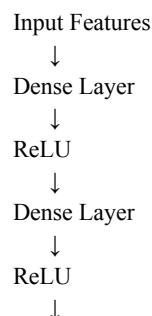
At least four conventional machine learning models and the MLP must be included.

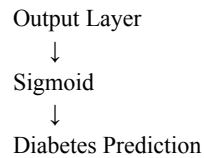
- ☐ Train the model.
- ☐ Generate predictions.
- ☐ Calculate evaluation metrics.
- ☐ Plot the confusion matrix.
- ☐ Record training time, where feasible.
- ☐ Analyze strengths and weaknesses.

12. Multilayer Perceptron Development

Develop an MLP specifically for diabetes prediction.

- Number of hidden layers
- Number of neurons per layer
- Activation functions
- Output activation
- Optimizer
- Learning rate
- Batch size
- Number of epochs





Students must justify their final architecture.

13. Model Training

- ☐ Train the MLP using the training data.
- ☐ Use a validation set or validation split.
- ☐ Record training and validation loss.
- ☐ Record training and validation performance.
- ☐ Plot learning curves.
- ☐ Identify signs of overfitting or underfitting.
- ☐ Experiment with appropriate regularization techniques.

Possible regularization techniques: Dropout, L2 regularization, Early stopping.

14. Model Evaluation

Required metrics:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- ROC-AUC
- ☐ ROC curve
- ☐ Precision-Recall curve
- ☐ Training/validation loss curve for the MLP

Students must discuss why accuracy alone is insufficient for evaluating a medical classification problem, with particular attention to false positives, false negatives, recall/sensitivity, precision, and F1-score.

15. Model Comparison and Selection

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression					
KNN					
SVM					
Random Forest					
MLP					

- Which model performed best?
- Which model achieved the highest recall?
- Which model achieved the highest ROC-AUC?
- Which model produced the fewest false negatives?
- Is the MLP necessarily the best model?
- Which model should be selected for deployment and why?

The final model must be selected using evidence from experiments.

16. Hyperparameter Tuning

Students must perform systematic hyperparameter tuning for the MLP.

Hyperparameter	Example Values
Number of hidden layers	1, 2, 3
Neurons	16, 32, 64, 128
Learning rate	0.001, 0.0005, 0.01
Batch size	16, 32, 64
Epochs	50, 100, 200
Activation	ReLU, Tanh
Optimizer	Adam, RMSprop
Dropout	0.0, 0.2, 0.3, 0.5

Students may use: Grid Search, Random Search, Bayesian-Based Hyperparameter Tuning (KerasTuner, Ray Tune or Optuna). The tuning strategy and search space must be documented.

17. Final Model Selection

- ☐ Select the best MLP configuration.
- ☐ Retrain the final model using the appropriate training data.
- ☐ Evaluate it on the unseen test set.
- ☐ Compare it against earlier baseline models.
- ☐ Report the final performance.
- ☐ Explain why the final architecture was selected.

Students should not tune hyperparameters using the test set.

18. Model Saving

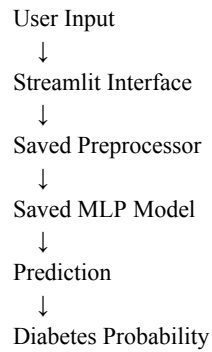
- Save the final trained model for deployment.
- TensorFlow/Keras:
`model.save("diabetes_mlp.keras")`
- Scikit-learn:
`import joblib`
`joblib.dump(model, "diabetes_model.pkl")`
- Students should also save the preprocessing/scaling object because the same transformation must be applied to future inputs.

19. Model Deployment

Deploy the selected model as a simple prediction application. The application should allow a user to enter patient attributes and obtain a prediction and probability.

Prediction: Diabetic / Non-Diabetic
Probability: XX.XX%

Tool	Suggested Use
Streamlit	Simple interactive ML web application
Gradio	Simple ML interface
Flask	REST-based web application
FastAPI	Production-style prediction API
Docker	Containerized deployment
Render/Railway	Cloud deployment
Hugging Face Spaces	Hosting interactive ML applications



Students should provide screenshots of the deployed application.

20. Testing the Deployed Model

- ☐ Test a likely non-diabetic case.
- ☐ Test a likely diabetic case.
- ☐ Test borderline cases.
- ☐ Verify that invalid inputs are handled appropriately.
- ☐ Verify that the application does not crash for unexpected inputs.
- ☐ Compare application predictions with Python notebook predictions.

21. Result Analysis

- Best-performing model
- MLP architecture
- Hyperparameters selected
- Final evaluation metrics
- Confusion matrix
- ROC curve
- Important features, where applicable
- Overfitting/underfitting analysis
- Comparison with baseline models
- Deployment performance

22. Limitations

- Dataset size and representativeness
- Potential demographic bias
- Data quality limitations
- False-positive and false-negative consequences
- Generalization to populations not represented in the dataset
- Why the model should not be treated as a standalone medical diagnostic system

23. Report format

Prepare the lab report with the following sections.

1. Project Title
2. Submitted by
 - i. Name (Regd No.:)
3. Team Members

- i. Name (Regd No.:)
 - ii. Name (Regd No.:)
 - iii. Name (Regd No.:)
- 4. Introduction
- 5. Problem Statement
- 6. Aim
- 7. Objectives
- 8. Dataset Description
 - i. Dataset Source
 - ii. Dataset Features
 - iii. Target Variable
 - iv. Dataset Statistics
 - v. Exploratory Data Analysis
 - vi. Dataset Structure
 - vii. Descriptive Statistics
 - viii. Missing Value Analysis
 - ix. Class Distribution
 - x. Univariate Analysis
 - xi. Bivariate Analysis
 - xii. Correlation Analysis
 - xiii. Outlier Analysis
- 9. Data Preprocessing
 - i. Data Cleaning
 - ii. Missing Value Handling
 - iii. Outlier Handling
 - iv. Feature Engineering
 - v. Feature Selection
 - vi. Train-Test Split
 - vii. Feature Scaling
- 10. Methodology
 - i. Overall Project Workflow
 - ii. Model Development Strategy
 - iii. Baseline Models
 - iv. Multilayer Perceptron Architecture
- 11. 11. Model Development
 - i. Logistic Regression
 - ii. K-Nearest Neighbors
 - iii. Support Vector Machine
 - iv. Random Forest
 - v. Multilayer Perceptron
- 12. 12. Model Evaluation
 - i. Accuracy
 - ii. Precision
 - iii. Recall
 - iv. F1-Score
 - v. Confusion Matrix
 - vi. ROC-AUC
 - vii. ROC Curve
 - viii. Precision-Recall Curve
- 13. Model Comparison and Selection
 - i. Performance Comparison
 - ii. Model Selection Criteria

- iii. Selected Model
- ii. Hyperparameter Tuning
 - i. Hyperparameters Considered
 - ii. Tuning Method
 - iii. Search Space
 - iv. Tuning Results
 - v. Optimized Architecture
- 14. Final Model Training
- 15. Final Model Evaluation
- 16. Model Deployment and Testing
 - i. Deployment Approach
 - ii. Deployment Tool
 - iii. User Interface
 - iv. Prediction Workflow
 - v. Deployment Testing
- 17. Results and Discussion
- 18. Limitations
- 19. Conclusion
- 20. Future Scope
- 21. References
- 22. Appendix
 - i. Source Code
 - ii. Additional Results
 - iii. Deployment Screenshots
 - iv. Contributions of Group Members

Sl. No.	Group Member	Regd. No.	Role / Responsibility	Contribution (%)
1				
2				
3				
4				

Students should indicate individual responsibility for areas such as:

- Dataset collection and understanding
- Exploratory Data Analysis
- Data preprocessing
- Feature engineering
- Baseline model development
- MLP development
- Hyperparameter tuning
- Model evaluation and comparison
- Deployment
- Report preparation

24. Final Deliverables

- i. Jupyter Notebook / Google Colab Notebook
- ii. Dataset or dataset access link
- iii. Preprocessing pipeline

- iv. EDA visualizations
- v. Multiple model implementations
- vi. MLP implementation
- vii. Hyperparameter tuning results
- viii. Model comparison table
- ix. Final trained model
- x. Deployment application/source code
- xi. Screenshots of deployment
- xii. Lab report

Key academic requirement: The objective is not merely to obtain the highest accuracy. Students must demonstrate the complete end-to-end deep learning workflow, justify their modeling decisions, compare alternatives, tune the selected model, and demonstrate its use through deployment.